

Aula 08 — Relações entre objetos

Quando um objeto precisa conhecer outro e quando essa relação faz parte da estrutura que ele deve manter?

PONTO DE PARTIDA

O modelo já contém relações

```
private Produto produto;          // em ItemPedido  
private List<ItemPedido> itens; // em Pedido
```

Esses campos guardam referências.

Mas qual responsabilidade torna cada referência necessária?

ATIVIDADE

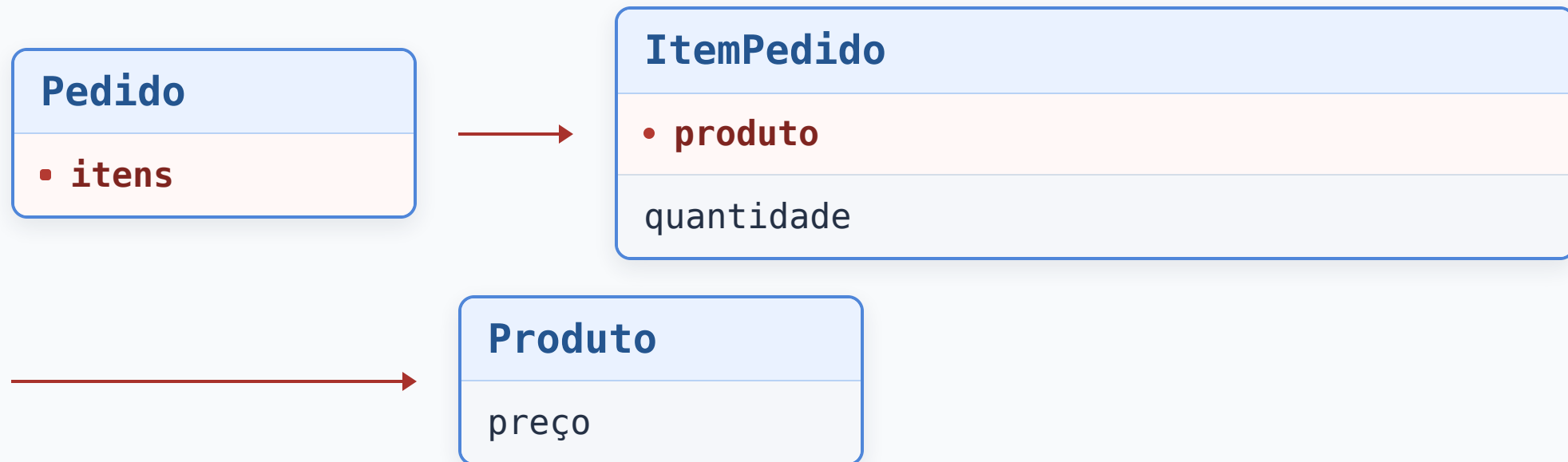
PONTO DE PARTIDA

Leia a direção das relações

No modelo atual:

1. quem precisa alcançar `Produto` para calcular um subtotal?
2. quem precisa alcançar vários `ItemPedido` para calcular um total?
3. `Produto` precisa conhecer pedidos para fornecer seu preço?

Cada referência resolve uma necessidade



- **Pedido** alcança os itens que coordena;
- **ItemPedido** alcança o produto que fornece o preço;
- **Produto** não precisa alcançar pedidos para cumprir seu papel atual.

CONCEITO-CHAVE

PONTO DE PARTIDA

Conceito-chave — associação entre objetos

Um objeto mantém uma referência para outro quando precisa colaborar com ele em alguma responsabilidade.

A conhecer B não implica que B também conheça A.

As duas relações têm o mesmo papel?

Relação	Por que existe?
ItemPedido → Produto	solicitar o preço necessário ao subtotal
Pedido → ItemPedido	reunir itens e coordenar o total

Produto pode existir no catálogo sem participar de pedido algum.

ATIVIDADE

ESTRUTURA DO PEDIDO

E se Pedido não mantivesse os itens?

Imagine que o campo `itens` desaparece.

1. quem teria de conhecer todos os itens para somar subtotais?
2. o que aconteceria com a ideia de que `Pedido` representa o conjunto?
3. o que um `Produto` perderia se não houvesse pedidos por enquanto?

O conjunto ficaria sem responsável

- algum código cliente teria de somar itens manualmente;
- a regra do conjunto ficaria fora de `Pedido` ;
- `Produto` continuaria existindo e mantendo suas próprias informações.

`Pedido` não consulta itens por acaso: ele mantém a estrutura formada por eles.

CONCEITO-CHAVE

ESTRUTURA DO PEDIDO

Conceito-chave — composição como responsabilidade estrutural

`Pedido` representa o todo e mantém os `ItemPedido` que formam sua estrutura.

`Produto` não é uma parte estrutural de um único pedido: pode ser usado em pedidos diferentes ou existir sem nenhum deles.

A criação em Main não decide tudo

```
ItemPedido item = new ItemPedido(teclado, 2);  
pedido.adicionarItem(item);
```

Criar o item antes de adicioná-lo pode ser útil para montar um cenário.

O papel conceitual continua sendo: o item representa uma parte de um pedido; o produto pode existir independentemente.

Manter a relação exige encapsulamento

```
public void adicionarItem(ItemPedido item) {  
    itens.add(item);  
}
```

`Pedido` oferece uma operação para receber itens.

Ele não precisa devolver sua lista interna para qualquer código modificar.

Toda relação reversa ajuda o modelo?

```
public class Produto {  
    private double preco;  
    private List<Pedido> pedidos;  
}
```

Que responsabilidade atual exige que um produto alcance todos os pedidos em que aparece?

Compare duas decisões

- **A.** Pedido mantém itens e cada item mantém seu produto.
- **B.** Além disso, cada produto mantém todos os pedidos em que aparece.

1. Qual responsabilidade atual exige B?
2. Qual decisão já permite calcular o total?
3. Quando uma relação reversa poderia fazer sentido?

Conhecimento adicional também tem custo

No modelo atual, nenhuma responsabilidade de **Produto** exige alcançar pedidos.

Uma relação adicional criaria perguntas novas:

- quem a atualiza?
- em que momento?
- como evitar que apenas um dos lados seja atualizado?

Dependência aparece nas chamadas

```
total += item.calcularSubtotal();
```

```
return produto.getPreco() * quantidade;
```

- Pedido precisa de calcularSubtotal();
- ItemPedido precisa de getPreco() .

Uma dependência aparece quando uma responsabilidade precisa de uma operação ou informação de outro objeto.

ARMADILHA

DIREÇÃO E NECESSIDADE

Armadilha — espelhar toda relação

Uma referência de A para B não deve ser duplicada em B apenas para tornar a estrutura simétrica.

A direção inversa só entra quando B também possui uma responsabilidade que exige alcançar A.

Uma biblioteca mantém empréstimos ativos

- **Livro** mantém título e autor;
- **Usuario** mantém seu nome;
- **Emprestimo** registra livro, usuário e data prevista;
- **Biblioteca** mantém empréstimos ativos.

Quais referências **Emprestimo** precisa manter? Qual relação é estrutural?

Uma estrutura e duas associações



- **Biblioteca** → **Emprestimo** é estrutural;
- **Emprestimo** → **Livro** e **Emprestimo** → **Usuario** permitem representar os participantes;
- livro e usuário existem independentemente do empréstimo ativo.

Síntese

- uma referência deve servir a uma responsabilidade;
- associações podem ter apenas uma direção;
- Pedido mantém a estrutura de itens que representa;
- Produto pode existir fora de pedidos;
- encapsulamento protege também as relações;
- dependências aparecem quando uma operação precisa de outra.

Antes de acrescentar uma relação, pergunte: quem precisa conhecer quem — e para fazer o quê?